

Les relations entre les classes

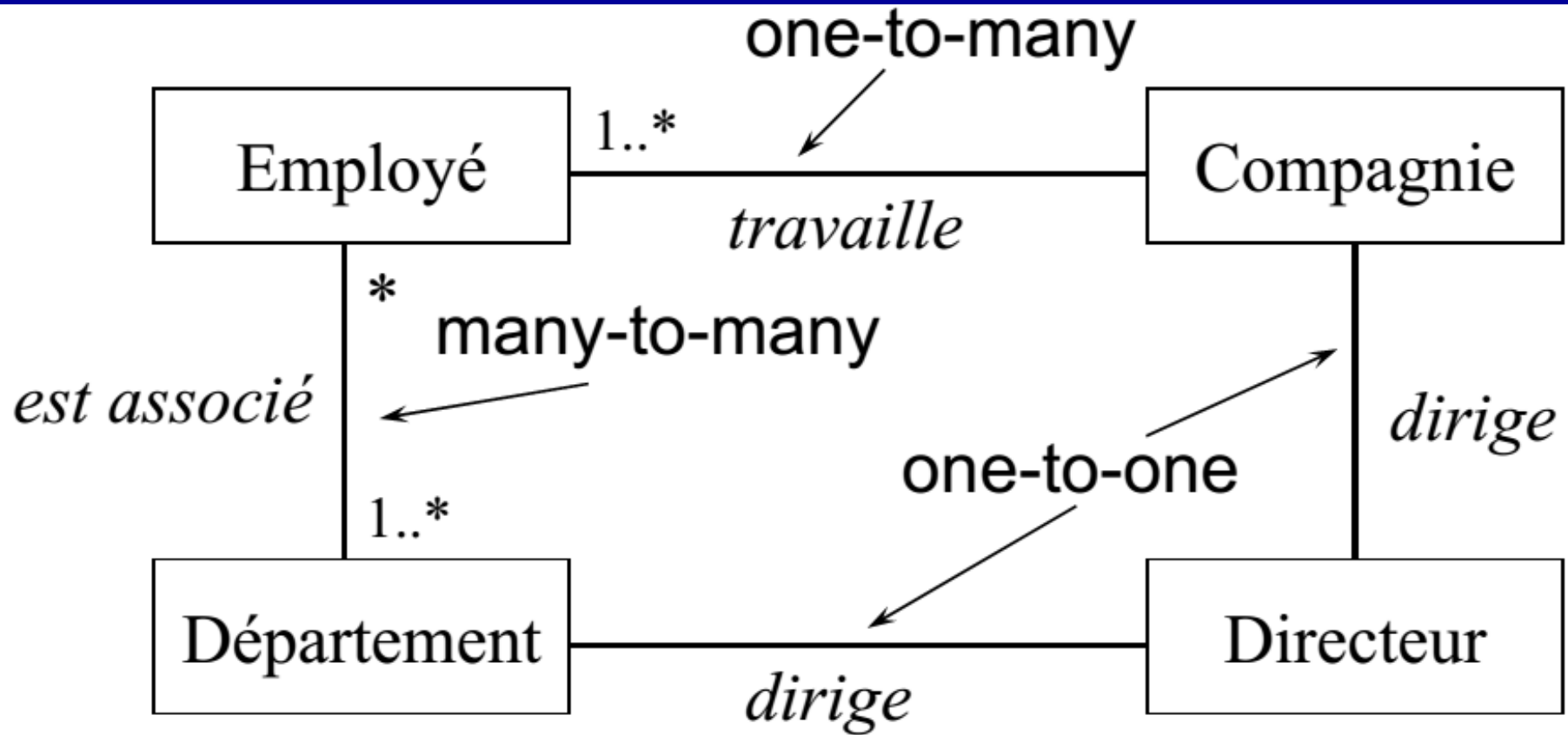
- Relations possibles entre les classes :
 - association,
 - agrégation et composition,
 - héritage simple et multiple,
 - dépendance.

Relation d'association

- La relation entre les classes la plus *générale*.
- Exprime l'existence d'un *lien* entre deux classes que l'on peut *nommer*.
- On complète cette relation en la *quantifiant*.

- 1 à 1 (one-to-one)
- 1 à plusieurs (one-to-many)
- Plusieurs à plusieurs (many-to-many)

Relation d'association



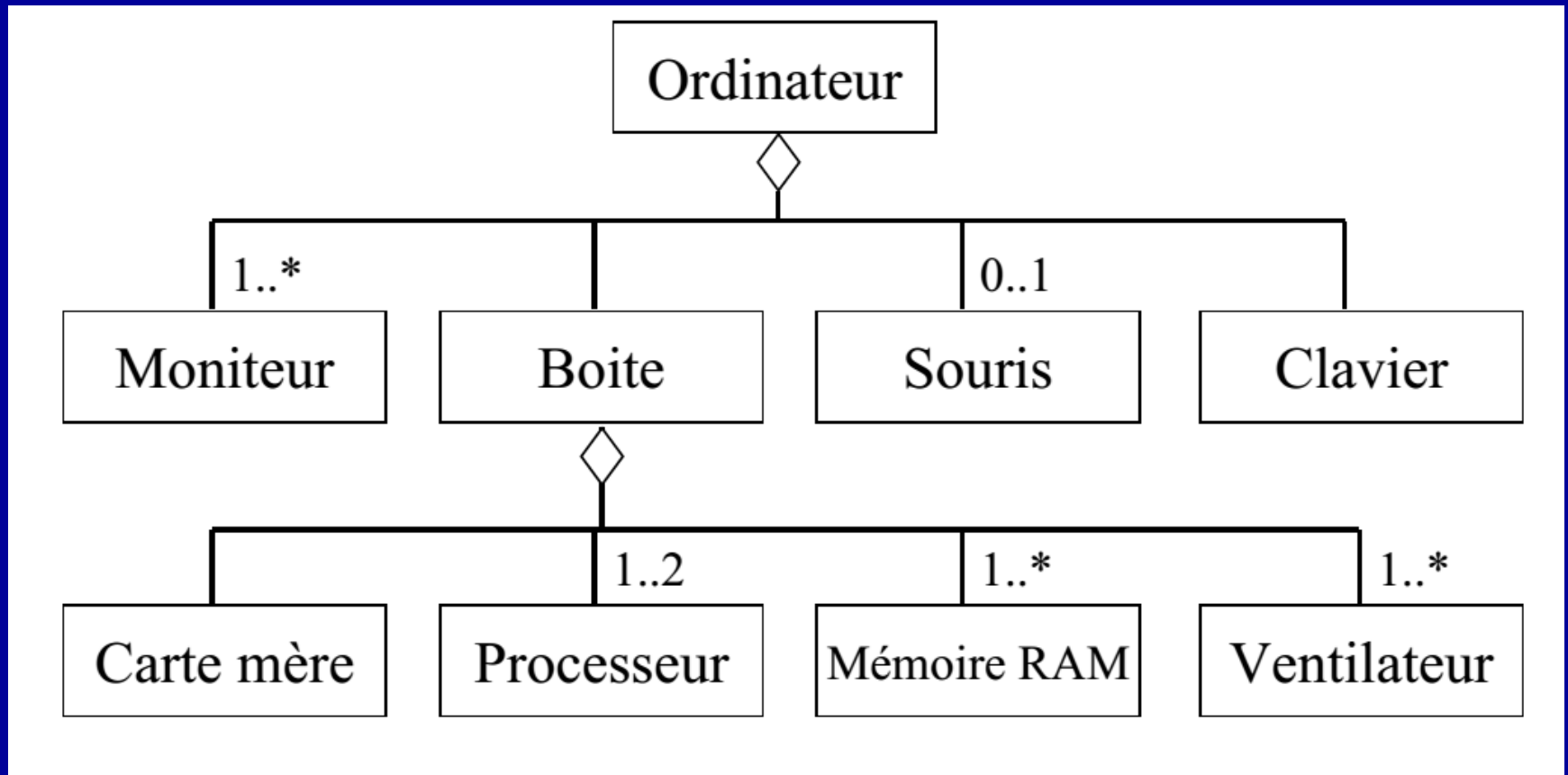
Composition vs agrégation

- Lien typique «du tout et de parties», **ou lien de «partie-de»**.
- Représentation des composants faisant partie d'un tout avec l'agrégation ou la composition.
- Relation transitive:
 - Si A fait partie de B et B fait partie de C, alors A fait partie de C.

Composition vs agrégation

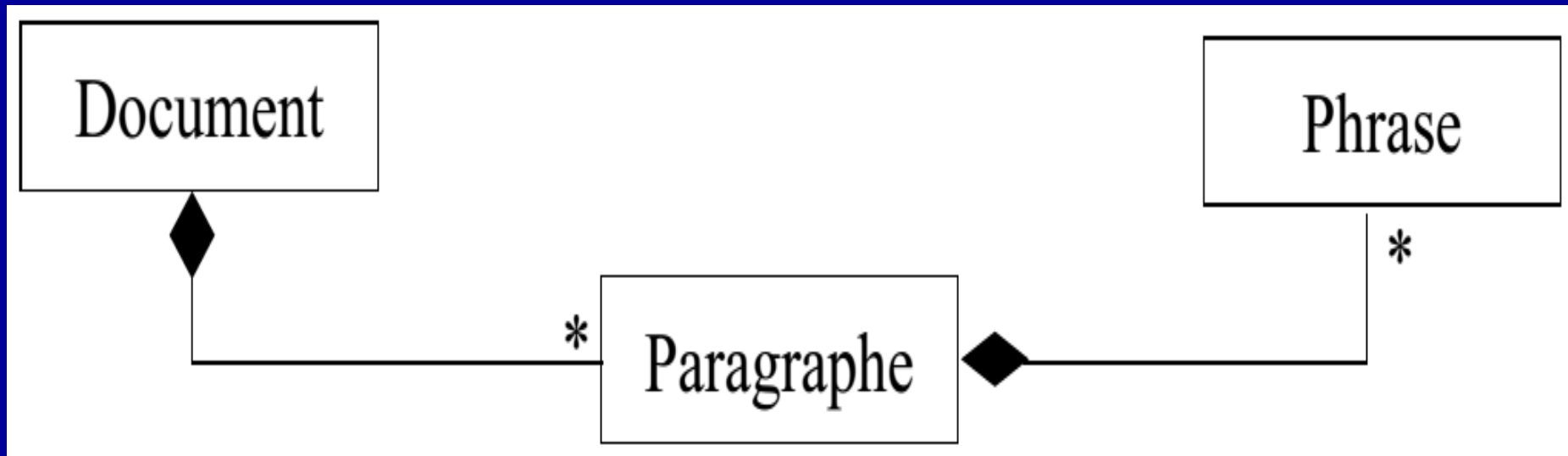
- La composition a un sens plus fort
 - si on élimine le tout, les parties ne peuvent exister en elles-mêmes.

Relation d'agrégation : exemple



Relation de composition :

- Exemple

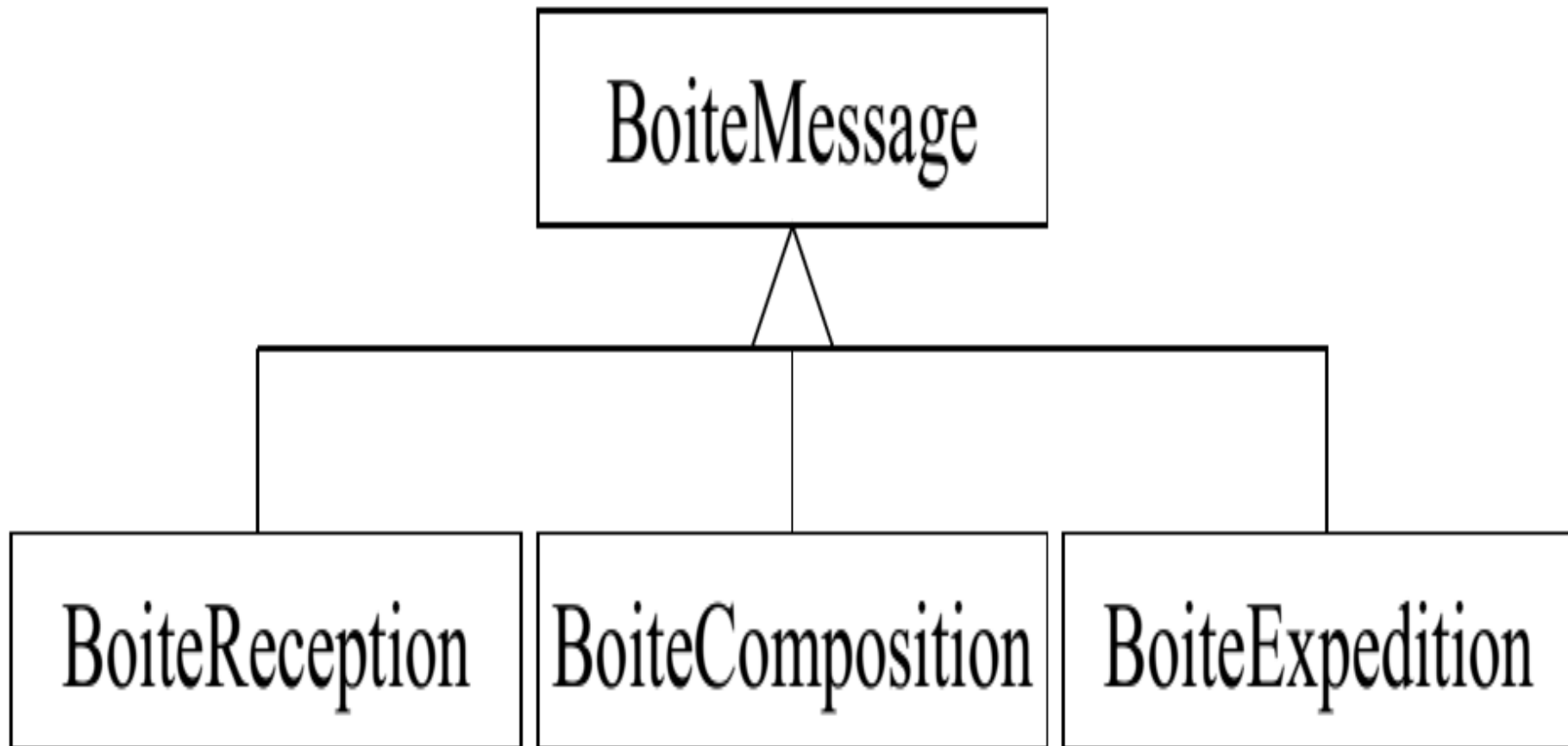


- Le document est composé de paragraphes qui sont à leur tour composés de phrases.
- Relation transitive,
 - i.e. le document est composé de phrases.
- Si on détruit le document, les paragraphes et les phrases sont détruits.

Relation d'héritage

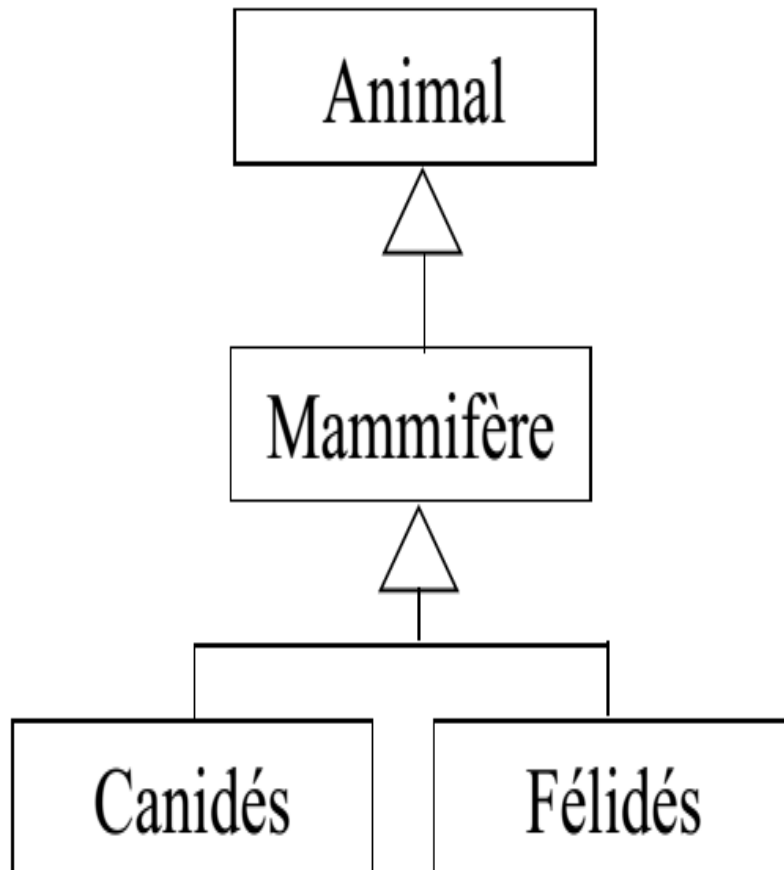
- Permet de partager les similitudes entre les classes tout en préservant leurs différences.
- La généralisation est souvent la relation «est-un»
 - une instance de la sous-classe est aussi une instance de la super-classe.
- Les méthodes et les attributs de la superclasse sont partagés avec toutes les sous-classes.

Relation d'héritage

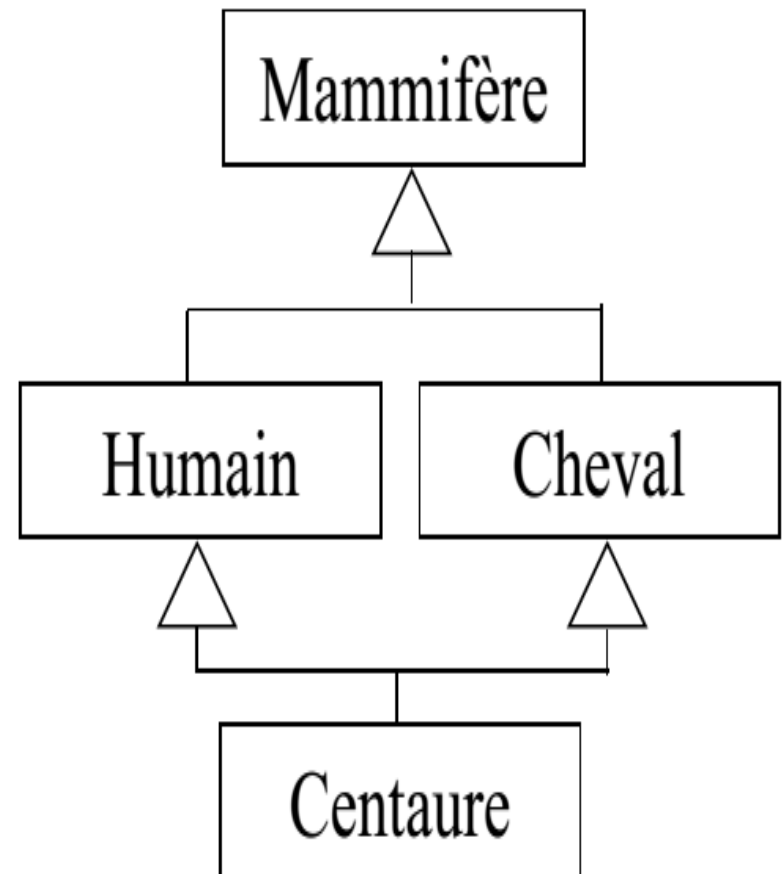


Héritage simple & multiple

Héritage simple



Héritage multiple



Héritage simple

```
class Mammifere
{ //... };
class Felide : public Mammifere
{ //... };
class Canide : public Mammifere
{ //... }
```

Héritage multiple

```
class Mammifere
{ //... };
class Cheval : public Mammifere
{ //... };
class Humain : public Mammifere
{ //... }
class Centaure : public Humain, public Cheval
{ //... };
```

Héritage multiple

- **Avantage:**
 - possibilités accrues de réutilisation
 - plus de flexibilité pour spécifier les classes
 - modélisation plus naturelle.
- **Désavantage:**
 - perte de simplicité conceptuelle ainsi que perte de clarté dans l'implémentation.
 - problèmes : hériter des mêmes attributs en plusieurs copies (deadly diamond).

Relation de dépendance

- Permet de modéliser des liens de service entre un client et un fournisseur de services.



- La fenêtre agit à titre de fournisseur de services permettant au dessin de s'afficher.
- Lien événementiel dans le temps.

Synthèse

- Relations

- 1

- 2

- 3

- 4

L'héritage

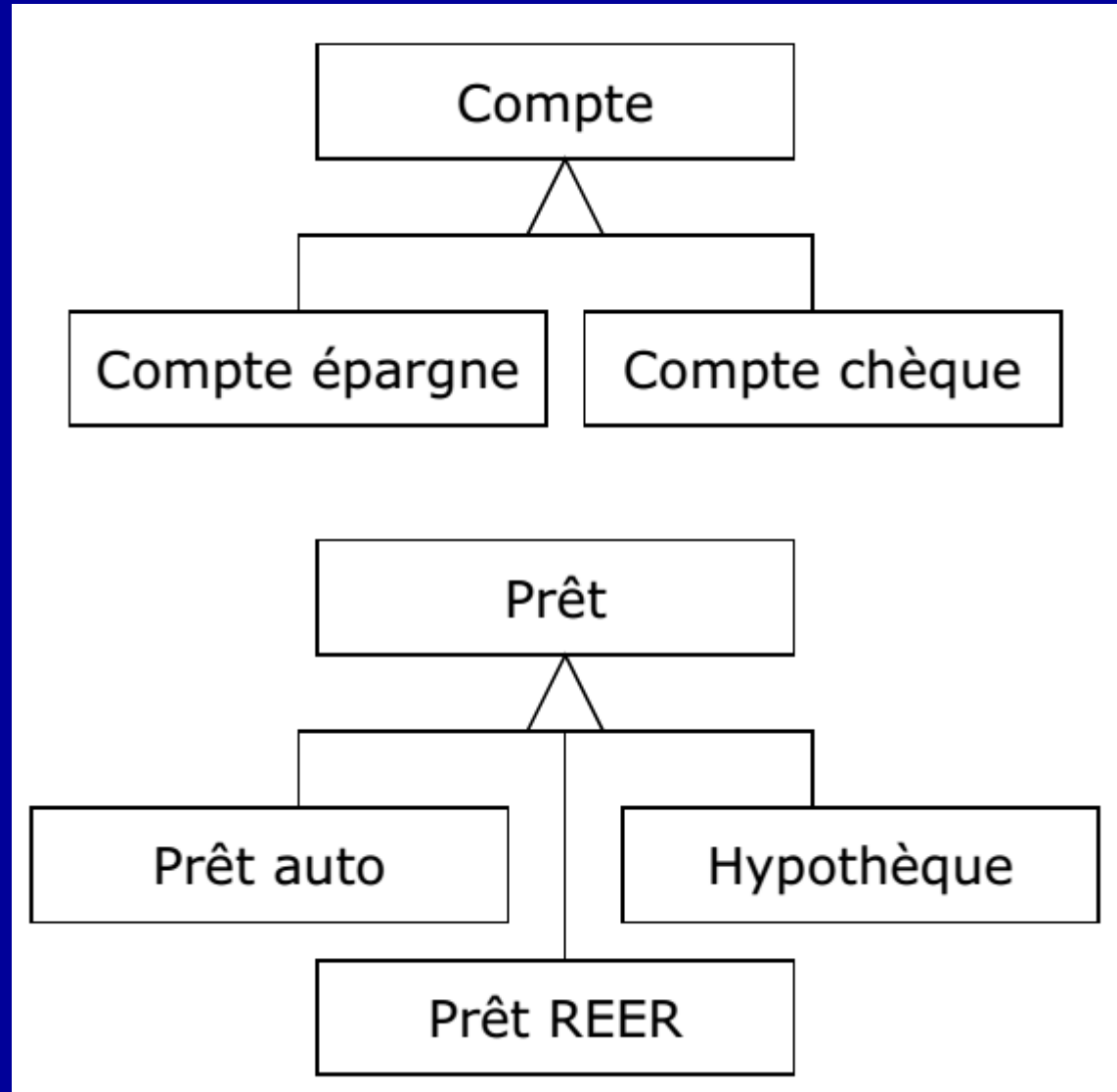
- Thème central de la programmation orientée objet.
- Fait la promotion d'une forme de réutilisation de logiciel
 - en créant de nouvelles classes à partir de classes existantes.
- Permet de sauver du temps de développement.
- Permet de réutiliser du code testé, débogué et de bonne qualité en général.

Classe de base et classe dérivée

- Dans le contexte d'héritage,
 - on dit souvent qu'un objet **est une sorte** d'un autre objet.
- Exemple :
- un Rectangle **est une sorte** de Quadrilatère.
 - Le Rectangle **hérite** d'un Quadrilatère;
 - La classe Quadrilatère est nommée la **classe de base**, et la classe Rectangle est nommée la **classe dérivée**.

Classe de Base et classe

- **Dérivée**
- Une hiérarchie prend la forme d'une structure d'arbre.
- Une classe de base existe dans une relation de hiérarchie avec les classes dérivées.

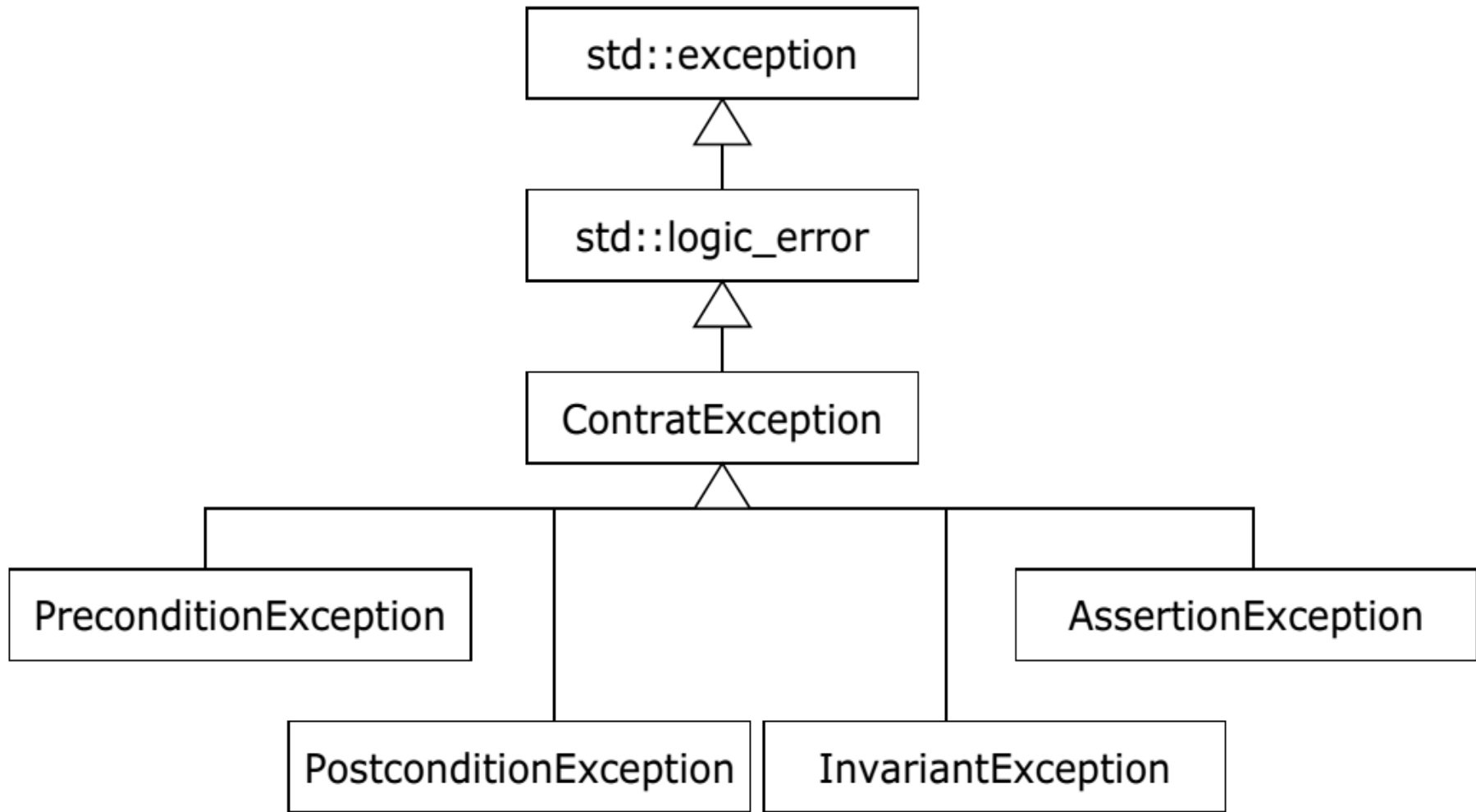


Héritage :constructeurs

- Si la classe de base dispose d'un constructeur par défaut,
 - appelé automatiquement lors de la création de l'objet de la classe dérivée.
- Si non
- appeler le constructeur de la classe de base de façon explicite pour lui passer les paramètres requis.

- obtenir les paramètres nécessaires à la construction de la classe de base à même les paramètres de la classe dérivée.
- Voir le code de ContratException
-

Hiérarchie des exceptions de la théorie du contrat



Définir la hiérarchie dans le fichier .h

Inclure la **classe dont on hérite**

```
#include <stdexcept>
class ContratException : public std::logic_error
{
public:
    ContratException(const char* fichierP, unsigned int noLigne,
                    const char* expressionP, const char* typeP);
    std::string      reqTexteException () const;

private:
    std::string      m_expression;
    std::string      m_fichier;
    unsigned int      m_ligne;
};
class PreconditionException : public ContratException
{
public:
    PreconditionException (const char* fichierP,
                        unsigned int noLigne,
                        const char* expressionP);
};
```

Établir le lien d'héritage

Établir le lien d'héritage

Définir la hiérarchie dans le fichier.cpp

```
#include "ContratException.h"
```

```
ContratException::ContratException(const char* fichierP,  
                                   unsigned int noLigne,  
                                   const char* expressionP,  
                                   const char* typeP)
```

Appeler le constructeur du parent

```
: std::logic_error(typeP), m_expression(expressionP),  
  m_fichier(fichierP), m_ligne(noLigne)
```

```
{  
}
```

Attributs de la classe courante

```
PreconditionException::PreconditionException(const char* fichierP,  
                                              unsigned int noLigne,  
                                              const char* expressionP)
```

```
: ContratException(fichierP, noLigne, expressionP,  
                  "ERREUR DE PRECONDITION")
```

```
{  
}
```

Appeler le constructeur du parent

Utiliser la hiérarchie création des objets

- Lorsqu'on crée un objet **PreconditionException**.
- il y a aussi création d'un objet **ContratException**.
- ainsi que d'un objet **std::logic_error** qui hérite lui même de **std::exception**.
- Il y a donc création automatique d'une classe de base lorsqu'on crée une classe dérivée qui se propage tout au long de la hiérarchie.

Construction et destruction

:Ordre d'appel

- Construction des objets de la hiérarchie
- de la classe de base tout en haut jusqu'à la classe dérivée.
- Destruction des objets de la hiérarchie
 - de la classe dérivée jusqu'à la classe de base.

Exemple : ordre d'appel des constructeurs et des destructeurs

```
class Base
{
public:
    Base (int db);
    ~Base ();
private:
    int m_b;
};

class Derive : public Base
{
public:
    Derive (int db, int dd);
    ~Derive ();
private:
    int m_d;
};

void main()
{
    Derive v (1234, 4321);
}
```

```
Base::Base (int db)
: m_b (db)
{
    cout << "Constructeur" << endl;
    cout << "Classe de base" << endl;
    cout << "Valeur : " << m_b << endl;
}
Base::~~Base()
{
    cout << "Destructeur" << endl;
    cout << "Classe de base" << endl;
}
Derive::Derive (int db, int dd)
: Base(db), m_d (dd)
{
    cout << "Constructeur" << endl;
    cout << "Classe derivee" << endl;
    cout << "Valeur : " << m_d << endl;
}
Derive::~~Derive()
{
    cout << "Destructeur" << endl;
    cout << "Classe derivee" << endl;
}
```

Héritage : contrôle des accès

- Les visibilités sont :
 - Privée : Accès interne de la classe.
 - Publique : Accès à tous les utilisateurs.
 - Protégée : Accès aux héritiers.
- Les méthodes d'une classe ont **toujours** accès aux attributs de la classe, peu importe la visibilité.

Héritage : contrôle des accès

- Les données privées ne sont manipulables que par les méthodes de la classe à laquelle les données appartiennent.
- L'héritage ne donne pas d'accès privilégié pour la visibilité privée.

Héritage : Contrôle des accès

- Les données protégées peuvent être manipulées par les classes héritières en plus des méthodes de la classe d'appartenance.
- Les utilisateurs de la hiérarchie n'ont pas accès aux données et/ou méthodes protégées.
 - Seules les méthodes et les attributs publics leur sont accessibles.

Exemple démontrant le contrôle des accès dans le contexte de l'héritage

```
#include <iostream>
using namespace std;
class A
{
public:
    A();
    void affiche() const;
    int m_i;

protected:
    int m_j;

private:
    int m_k;
};
```

```
A::A()  
: m_i(1), m_j(2), m_k(3)  
{  
}  
  
void A::affiche() const  
{  
    cout << "i : " << m_i << endl;  
    cout << "j : " << m_j << endl;  
    cout << "k : " << m_k << endl;  
}
```

Exemple démontrant le contrôle des accès dans le contexte de l'héritage

```
class B : public A
{
public:
    void modifieValeurs();
};
```

```
void B::modifieValeurs ()
{
```

```
    m_i = 11;        // --- Possible m_i est d'accès publique dans la classe A
    m_j = 12;        // --- Possible m_j est d'accès protégée dans la classe A
    //m_k = 13;      // --- Impossible m_k est d'accès privée dans la classe A
    cout << "m_i et m_j ont ete modifiees!" << endl;
```

```
}
```

```
void main()
```

```
{
```

```
    B unB;
    unB.affiche();
    unB.modifieValeurs();
    unB.affiche();
    //unB.m_j = 7; // --- Impossible m_j est d'accès protégée
```

```
}
```

Résultat:

i:1

j:2

k:3

m_i et m_j ont ete modifiees!

i:11

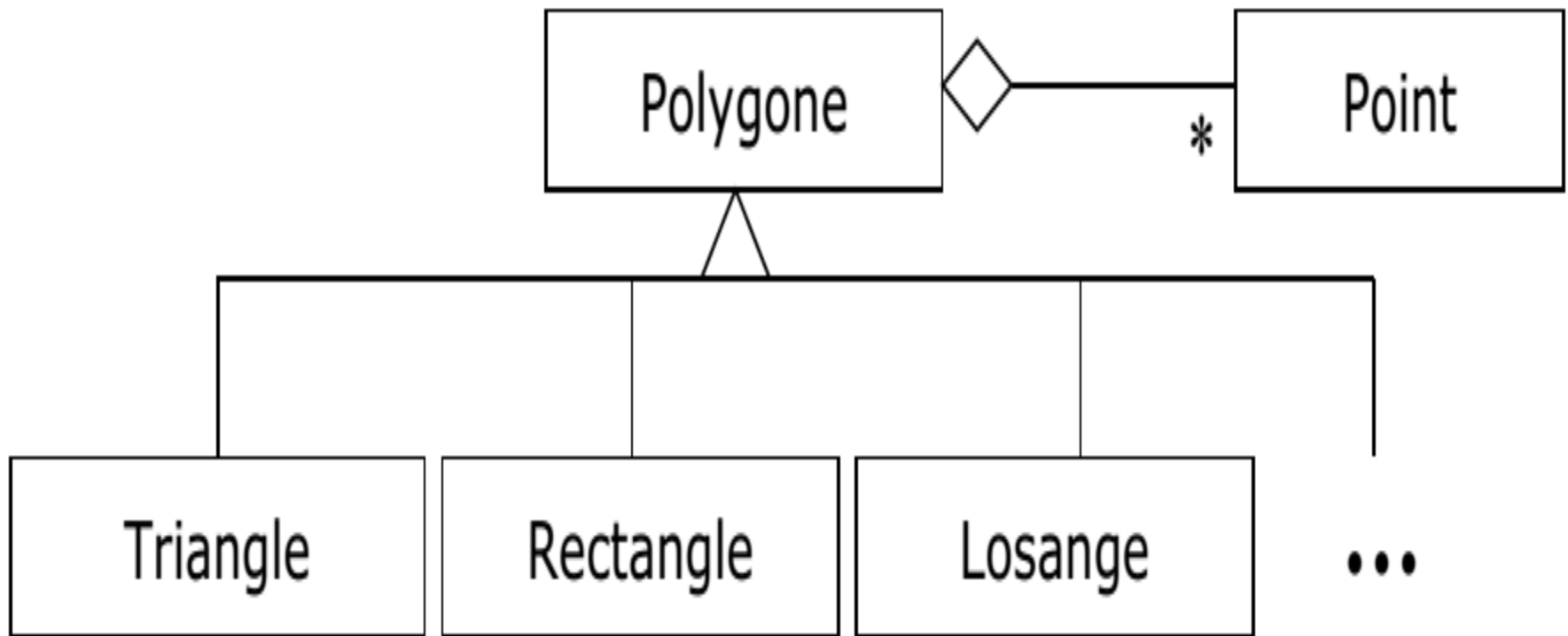
j:12

k:3

- Déf:
- 2 types de classes:
- Implantation:
- Construction:
- Destruction:
- Accès:

Héritage: Exemple

- Figures géométriques.



Héritage :Exemple ...

Classe Point utilisée par les polygones:

```
class Point
{
public:
    Point      (int x=0, int y=0);
    void  deplacer (int dx, int dy);
    void  agrandir (const Point& centre, double facteur);
    void  afficher (std::ostream& os) const;
    double x      () const;
    double y      () const;

private:
    double m_x;
    double m_y;
};

void Point::afficher(std::ostream& os) const
{
    os << "[" << m_x << ", " << m_y << "];"
}
```

Héritage : Exemple ...

- Classe Polygone : classe de base à partir de laquelle on pourra faire de l'héritage :
-

```
class Polygone
{
public:
    Polygone      (int nbSommets = 0);
    void          asgPoint  (int i, const Point& p);
    void          deplacer  (int dx, int dy);
    void          agrandir  (const Point& centre,
                           double facteur);
    void          afficher  (std::ostream& os) const;
protected:
    const Point& reqPoint   (int pos) const;
private:
    std::vector<Point> m_vPoints;
};
```

Héritage :Exemple ...

- La méthode **afficher** imprime le nombre de sommets et les coordonnées de chacun des points:

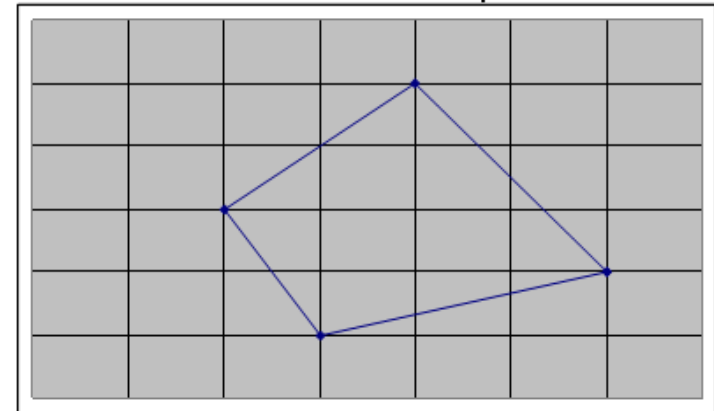
```
void Polygone::afficher(ostream& os) const
{
    int dim = m_vPoints.size();
    os << "Polygone: " << dim;
    for (int i=0; i<dim; i++)
    {
        os << " " << m_vPoints[i].afficher(os);
    }
}
```

Le **point** a une méthode **afficher** pour imprimer ses coordonnées.

Héritage :Exemple ...

- Ainsi, notre polygone pourrait imprimer:

```
Polygone q(4);  
q.asgPoint(1,Point(3.0, 2.0));  
q.asgPoint(2,Point(1.5, 2.5));  
q.asgPoint(3,Point(1.0, 1.5));  
q.asgPoint(4,Point(2.0, 0.5));  
q.afficher(cout);
```



Résultat :

```
Polygone 4 [3, 2] [1.5, 2.5] [1, 1.5] [2, 0.5]
```

Que pensez-vous de l'interface du polygone ?

Héritage :Exemple ...

- Supposons maintenant que l'on veuille créer une classe **Triangle**.
- Hériter de la classe **Polygone**
 - Le Triangle «est un» Polygone.
 - Cas **concret** de polygone.
-

- Le triangle se comporte de la même façon que le polygone, excepté pour sa construction(où on lui fournit exactement trois sommets).

- ```
class Triangle : public Polygone
{
public:
 Triangle(const Point& p1,
 const Point& p2,
 const Point& p3);
};
```

# Héritage :Exemple ...

- écrire le constructeur de Triangle dans lequel nous devons construire la classe de base, le Polygone:

```
Triangle::Triangle(const Point& p1,
 const Point& p2,
 const Point& p3)
```

```
: Polygone (3)
{
 asgPoint(1, p1);
 asgPoint(2, p2);
 asgPoint(3, p3);
}
```

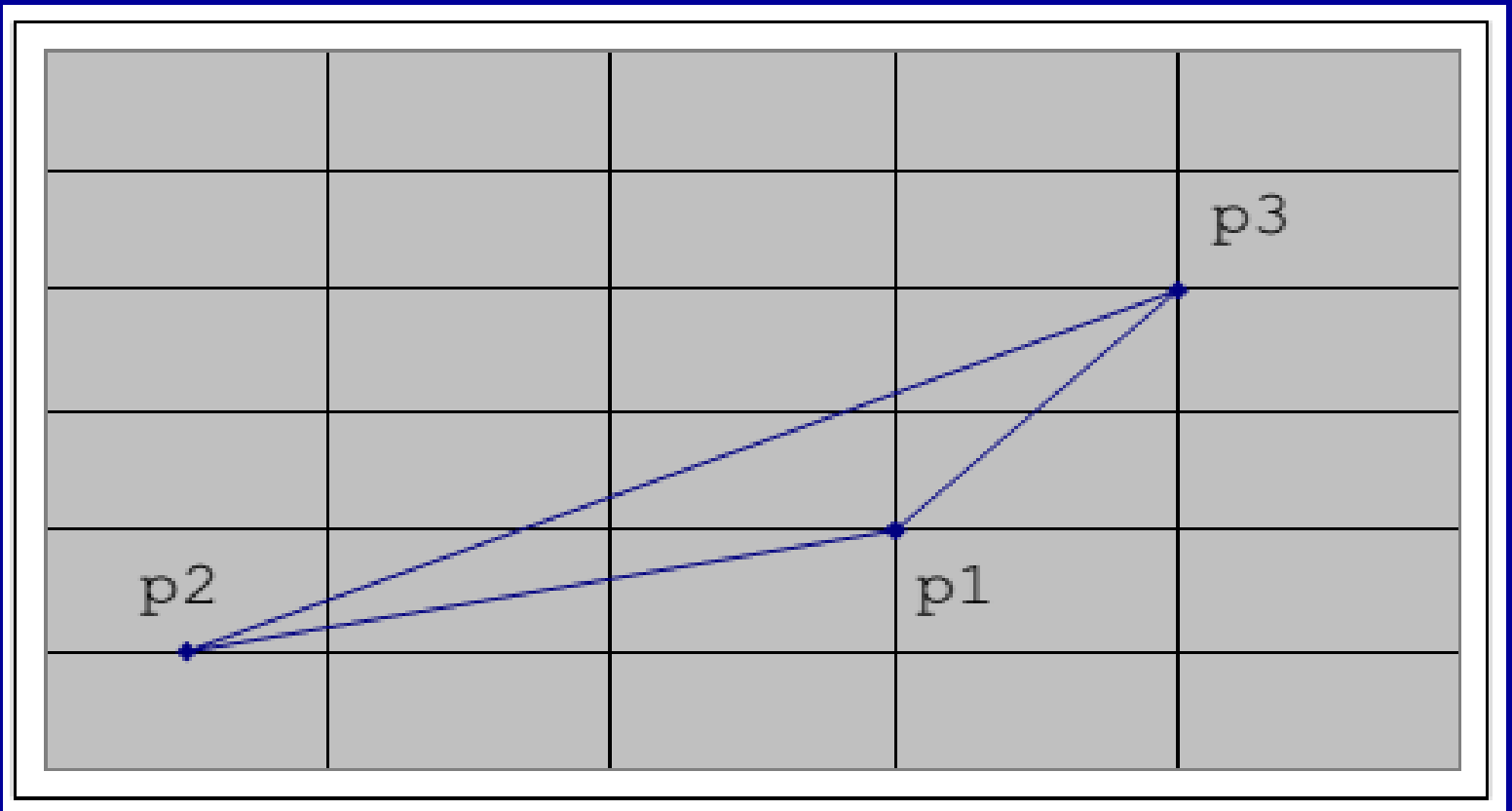
Est-ce que le **triangle**  
est valide dès sa  
création ?

# Héritage :Exemple ...

- Toutes les méthodes publiques de la classe Polygone sont directement accessibles aux utilisateurs du Triangle  
Le Triangle «est un» Polygone :

# Le Triangle «est un» Polygone :

```
void main()
{
 Point p1 (3.0, 2.0) ;
 Point p2 (0.5, 2.5) ;
 Point p3 (4.0, 1.0) ;
 Triangle t (p1, p2, p3) ;
 t.afficher(cout) ;
}
```



**Résultat :**

Polygone 3 [3, 2] [0.5, 2.5] [4, 1]

# Héritage *vs* Composition

- **composition:**

```
class Triangle2
{
public:
 Triangle2 (const Point& p1,
 const Point& p2,
 const Point& p3);
 void afficher (ostream& os) const;

private:
 Polygone m_polygone;
};
```

# Héritage *vs* Composition

- écrire le constructeur de Triangle2 dans lequel nous devons construire notre attribut **m\_polygone** et non la classe de Base:



```
Triangle2::Triangle2(const Point& p1,
 const Point& p2,
 const Point& p3)
: m_polygone(3)
{
 m_polygone.asgPoint(1, p1);
 m_polygone.asgPoint(2, p2);
 m_polygone.asgPoint(3, p3);
}
```

# Héritage *vs* Composition

- La méthode *afficher* doit être réécrite pour être accessible à la nouvelle classe.
- La réutilisation par composition est une autre forme de réutilisation souvent utilisée.

# Héritage *vs* Composition

```
void Triangle2::afficher(ostream& os) const
{
 // --- On redirige simplement l'appel vers le
 // --- polygone.
 m_polygone.afficher(os);
}
```

# Exercices

- Coder une classe **Rectangle** se construisant avec deux **Points**, i.e. les coins opposés :
  - en utilisant la **composition**.
  - en utilisant l'*héritage* (hérite de **Polygone**).

p1



p2

# Ajout de méthodes dans la classe de base

- On veut ajouter `calculerCirconference` dans la classe `Polygone`
  - calcule la circonférence d'un polygone en ajoutant la longueur de chacun des segments de droite entre les sommets.



# Ajout de méthodes dans la classe de base

- L'algorithme du calcul de circonférence devient :
  - faire la somme des segments du polygone ...
  - (Ne pas oublier de tenir compte du dernier point qui relie le premier point)



# Ajout de méthodes dans la classe de base

- Utilité d'avoir une méthode de calcul de la distance entre deux points `calculerDistance`:
  - À quelle classe doit-on assigner cette responsabilité ?
  - Quel est l'algorithme ?

# Ajout de méthodes dans la classe de base

1. calcul de la distance entre deux points :

□ La classe Point est le candidat idéal pour avoir cette responsabilité.

□ Algo :  $d = \sqrt{(p2.x - p1.x)^2 + (p2.y - p1.y)^2}$

```
double Point::calculerDistance(const Point& p1,
 const Point& p2)
{
 double x2 = pow(p2.x() - p1.x(), 2);
 double y2 = pow(p2.y() - p1.y(), 2);
 return sqrt (x2 + y2);
}
```

C'est une méthode **statique**.

# Ajout de méthodes dans la classe de base

## 2. La méthode **calculerCirconference** devient

```
double Polygone::circonference() const
{
 double circonference = 0;
 for (int i=0; i<m_vPoints.size(); i++)
 {
 Point p1 = m_vPoints[i];
 Point p2 = (i+1 == m_vPoints.size()) ?
 m_vPoints[0] : m_vPoints[i+1];
 circonference += Point::calculerDistance(p1, p2);
 }
 return circonference;
}
```

# Ajout de méthodes

**Triangle** qui est une classe dérivée de **Polygone**

- A *immédiatement accès* à cette nouvelle fonctionnalité,

**Triangle2** qui utilise **Polygone**

- doit être ajoutée manuellement dans la classe **Triangle2**.

# Ajout de méthodes

- ✓ Résultat intéressant d'un point de vue de gestion du code.
- tout héritier de la classe Polygone a accès à ses méthodes qu'elles soient écrites ***maintenant*** ou dans le ***futur***.

Remplacer, hériter, augmenter